

Relatório de Melhorias — Fábrica de Materiais de Comunicação

Estudo de caso: produção do Kit inLego (/esbocar → /produzir-comunicacao-completa)

Auditoria técnica do pipeline

2026-08-05

Sumário

Relatório de Melhorias — Fábrica de Materiais de Comunicação	1
0. Erro no fluxo — “Textos” não foi gerado	1
1. Pontos de bug ou correção (mais crítico → menos crítico)	3
● Crítico	3
● Alto	3
● Médio	3
● Baixo	4
2. Pontos de melhoria e refinamento	4
Commands	4
Rules (CLAUDE.md)	4
Specs	5
Skills	5
Subagentes	5
Hooks	5
MCPs	5
3. Pontos de otimização de tokens	6
Onde a redundância é real	6
Avaliação honesta da proposta do usuário (grafo de insumos via code-review-graph) ...	6
Otimizações concretas propostas	6
Resumo executivo	7

Relatório de Melhorias — Fábrica de Materiais de Comunicação

Estudo de caso: execução completa de /esbocar Kit inLego seguida de /produzir-comunicacao-completa kit-inlego, com auditoria de Skills, Rules, Specs, MCPs, Subagentes, Hooks e Commands do projeto proj_fabrica-comunicacao.

0. Erro no fluxo — “Textos” não foi gerado

Sintoma relatado: os textos de apoio para WhatsApp/Instagram/LinkedIn, fixados anteriormente como 7º material de primeira classe do pipeline, não foram gerados para o Kit inLego.

Causa raiz confirmada (com evidência de `git log/git show`):

O material textos foi integrado ao pipeline em dois commits:

Commit	O que fez
95ccf5c — “ <i>add texts for whatsapp, instagram, and linkedin as a 7th first-class material</i> ”	Criou <code>.claude/skills/redator-textos/SKILL.md</code> , <code>scripts/validar-textos.py</code> ; adicionou “textos” a <code>TIPOS_VALIDOS</code> em <code>scripts/parametros_projeto.py</code> e <code>scripts/auditar-projeto.py</code> .
09bb74f — “ <i>integrate ‘textos’ material in SPEC, subagent definitions, and orchestration commands</i> ”	Criou <code>.claude/agents/subagente-produtor-textos.md</code> ; atualizou <code>.claude/commands/produzir-comunicacao-completa.md</code> (dispatch de textos → <code>subagente-produtor-textos</code>); atualizou <code>SPEC.md</code> para descrever textos como 7º material selecionável (“1 a 7 selecionados”).

Nenhum dos dois commits tocou `.claude/commands/esbocar.md`. O Passo 4 desse comando — a única interface humana que decide `materiais_selecionados` — continua hardcoded com o texto literal “*Uma única pergunta, multiSelect, com as 6 opções*”, listando só PDF/Landing/Apresentação/Arte-01/02/03. `git log --all` confirma que `esbocar.md` **nunca**, em nenhum commit, teve uma opção “Textos de Apoio” na Rodada 4.

Consequência prática: o material textos está 100% funcional do meio para o fim do pipeline (skill, subagente, validador, dispatcher, spec) mas é **estruturalmente inatingível** a partir da única porta de entrada humana. Nenhum operador jamais conseguirá selecioná-lo via `/esbocar` como o comando existe hoje — não é uma falha de execução desta sessão, é uma lacuna permanente até que `esbocar.md` seja corrigido.

(Achado colateral, menor: `output/kit-stop-drill/config_projeto.json` — um projeto anterior não versionado no git — já contém “textos” em `materiais_selecionados`, provavelmente inserido manualmente/por edição direta do JSON, não pela entrevista. Reforça que a única via viável hoje para incluir textos é contornar `/esbocar`.)

Correção recomendada (crítica, baixo esforço): adicionar a 7ª opção “Textos de Apoio (WhatsApp/Instagram/LinkedIn)” à Rodada 4 de `.claude/commands/esbocar.md`, subindo a contagem de “6 opções” para “7 opções” no texto do comando. Ver também a recomendação de teste de consistência automatizado na Seção 2, que teria pego essa lacuna antes de chegar à produção.

1. Pontos de bug ou correção (mais crítico → menos crítico)

● Crítico

1.1 — textos inatingível via /esbocar. Ver Seção 0. Único ponto de falha, correção de poucas linhas em `esbocar.md`.

● Alto

1.2 — `scripts/compilar-html.py::compilar_apresentacao` com path de imagem hardcoded. Corrigido durante esta sessão. A função copiava/injetava `insumos/kit_start_flex_frontal.png` incondicionalmente — um resíduo do projeto `kit-start-flex`. Qualquer projeto com `apresentacao` selecionada, cujo slug não fosse `kit-start-flex`, receberia uma imagem de produto ausente/quebrada no slide de capa. `compilar_landing` no mesmo arquivo já tinha sido corrigido por um subagente anterior (resolve o path via `config_projeto.json.imagens[0].path`) — `compilar_apresentacao` não recebeu o mesmo tratamento até esta sessão. **Correção:** aplicado o mesmo padrão de resolução dinâmica via `config_projeto.json`.

1.3 — `scripts/compilar-arte.py` com o mesmo bug de path de imagem hardcoded. Mesma causa e mesmo risco do item 1.2, afetando as 3 variantes de arte. Corrigido com o mesmo padrão nesta sessão.

1.4 — `scripts/compilar-arte.py` deletava o `index.html` temporário exigido pelo validador de marca. O script sempre apagava o HTML usado pelo Playwright logo após o screenshot, mas `scripts/validar-design-tokens.py` exige `output/<slug>/<tipo>/index.html` persistido para checar cores fora do design system fixo. Isso forçou **3 subagentes de revisão de marca diferentes** (`arte-01`, `arte-02`, `arte-03`) a reconstruir manualmente o HTML como workaround local, cada um gastando ciclos de diagnóstico para redescobrir o mesmo problema. **Correção:** `index.html` agora é persistido (não mais deletado) após a renderização bem-sucedida.

1.5 — `scripts/compilar-pdf.py` com título, subtítulo, CTA e imagem de capa hardcoded para `kit-start-flex`. Encontrado e corrigido pelo subagente de PDF durante esta execução — sem a correção, qualquer outro projeto receberia o título, CTA e capa errados no PDF final. Generalizado para derivar esses valores do dossiê e de `config_projeto.json`.

● Médio

1.6 — Cobertura do grafo de conhecimento (`code-review-graph`) muito abaixo do que a REGRA 9 do CLAUDE.md pressupõe. O grafo indexa apenas os scripts Python do pipeline (17 arquivos, confirmado pelo hook de `SessionStart`: “*Nodes: 72, Edges: 973, Files: 17, Languages: python, bash*”) — **zero cobertura** de `.claude/skills/*` (16 arquivos), `.claude/agents/*.md` (6), `.claude/commands/*.md` (6) e dos `SPEC*.md/ CLAUDE.md`. A REGRA 9 manda “todo agente/skill” ser localizado primeiro pelo grafo antes de qualquer leitura de arquivo — mas para a maior parte do que compõe a fábrica (skills, agentes, comandos, specs), o grafo literalmente não tem nós para responder, forçando fallback silencioso para leitura direta de qualquer forma. Além disso, a busca semântica não tem embeddings gerados nesta instância — cai para busca textual literal, o que enfraquece a promessa de “busca semântica” do MCP.

1.7 — Documentação de hooks desalinhada com o mecanismo real ativo. O `CLAUDE.md` atribui a atualização automática do grafo aos hooks em `.gemini/hooks/crg-update.sh` e `.gemini/hooks/crg-session-start.sh`. Esses arquivos existem e funcionam, mas o próprio comentário dentro de `crg-update.sh` diz “*Gemini CLI hook*” — ou seja, são escritos para o Gemini CLI, não para o Claude Code. O mecanismo que de fato dispara neste ambiente (Claude Code) é a configuração `PostToolUse/SessionStart` em `.claude/settings.json`, que roda comandos equivalentes (`code-review-graph update/status`) mas não é mencionada em nenhum lugar do `CLAUDE.md`. Um mantenedor tentando debugar ou estender o comportamento do grafo, seguindo a documentação, editaria o arquivo errado.

● Baixo

1.8 — `output/kit-stop-drill/config_projeto.json` referencia a imagem oficial do produto com o nome de arquivo `kit_start_flex_frontal.png`, mesmo a descrição dizendo “foto oficial do kit stop drill”. Não quebra nada hoje (o arquivo existe com esse nome em disco), mas é um nome de arquivo enganoso — provavelmente um artefato de quando esse projeto foi montado copiando convenções do `kit-start-flex`. Este projeto não está versionado no git, então não é possível confirmar se veio de uma versão mais antiga do fluxo ou de edição manual.

1.9 — `.claude/settings.local.json` acumula permissões de Bash literais e escapadas a `kit-start-flex` (ex.: `Bash(python scripts/validar-pdf.py kit-start-flex)`). Isso não generaliza: cada novo projeto (`kit-inlego`, futuros slugs) vai gerar prompts de permissão repetidos para comandos estruturalmente idênticos, em vez de casar com um padrão wildcard.

1.10 — Scripts utilitários de debug (`.crg-regenerate.py`, `.crg-visual.py`, na raiz do repo) hardcodam `output/kit-start-flex` como alvo. Não fazem parte do pipeline formal (prefixo `..`, fora de `scripts/`), mas rodá-los sem editar apontaria silenciosamente para um projeto errado/desatualizado.

2. Pontos de melhoria e refinamento

Commands

- **Checklist de consistência automatizada entre `esbocar.md` e o restante do pipeline.** O bug da Seção 0 só existe porque nada verifica, de forma determinística, que todo tipo em `TIPOS_VALIDOS` (`parametros_projeto.py/auditar-projeto.py`) também aparece (a) como opção selecionável no Passo 4 de `esbocar.md`, (b) no mapa de dispatch de `produzir-comunicacao-completa.md`, (c) com um `redator-*` e um `subagente-produtor-*` correspondentes. Um script `scripts/verificar-consistencia-pipeline.py`, rodado antes de qualquer commit que toque nesses arquivos (ou via hook de `PreToolUse/pre-commit`), teria pego essa lacuna no dia em que “textos” foi parcialmente integrado, em vez de só ser descoberta na produção real de um projeto.

Rules (`CLAUDE.md`)

- **REGRA 9 precisa refletir a cobertura real do grafo** (achado 1.6) — ou expandir o escopo de indexação do `code-review-graph` para incluir `.md` de `skills/agentes/specs` (as tools `get_docs_section_tool`, `generate_wiki_tool`, `get_wiki_page_tool` sugerem que

o MCP já tem uma trilha de documentação que talvez nunca tenha sido rodada via `run_postprocess_tool/generate_wiki_tool` neste repo), ou reduzir explicitamente o mandato da REGRA 9 a “código e scripts”, deixando claro que `skills/agentes/specs` continuam sendo descobertos por busca direta — evita que um agente gaste uma chamada de ferramenta em uma busca semântica que estruturalmente não pode responder.

- **Seção “O grafo se auto-atualiza” precisa documentar os dois mecanismos** (achado 1.7): `.claude/settings.json` (ativo no Claude Code) e `.gemini/hooks/*.sh` (ativo no Gemini CLI), deixando explícito qual é autoritativo em qual ambiente.

Specs

- **Tabela de materiais em `SPEC.md`/`CLAUDE.md` deveria ter uma coluna “selecionável via / esbocar”** — tornando explícito, na própria tabela de contrato, que todo material com redator-`*/subagente-produtor-*/validar-*.py` precisa também constar na entrevista. Isso é o mesmo achado da Seção 0 e do item de Commands acima, mas também vale como reforço documental, não só como script de verificação.

Skills

- **`aplicador-marca-conexao/SKILL.md` poderia publicar um digest curto e versionado das variáveis-chave de `brand/design-system-conexao.json` (cores, nomes de gradiente) diretamente no corpo do skill**, para que os `compilador-*` não precisem reabrir o JSON completo repetidamente — o arquivo é idêntico entre projetos, então cachear/resumir esse conteúdo específico não fere a REGRA 7 (que protege insumos e estado de pipeline, não o design system fixo).

Subagentes

- Nenhum conflito de instrução direto foi encontrado entre os `subagente-produtor-*` e `subagente-revisor-marca` nesta rodada — a divisão de responsabilidade (produtor não invoca `revisor-marca`; revisor só atua depois que o lote termina) está consistente entre todos os arquivos lidos. O ganho aqui é de processo, não de correção de texto: ver a otimização de pre-flight na Seção 3, que evitaria que o mesmo bug de infraestrutura seja “descoberto” de forma redundante por múltiplos subagentes de produção/revisão em paralelo.

Hooks

- Já coberto acima (achado 1.7): alinhar documentação aos dois mecanismos reais.
- Adicionalmente, nenhum tratamento de erro visível nos scripts de hook (`crg-update.sh/crg-session-start.sh`) além do `|| true` genérico — aceitável para um hook de “melhor esforço” que não deve travar o fluxo principal, mas isso significa que uma falha real de indexação do grafo é sistematicamente silenciosa. Considerar logar falhas (não bloquear) em vez de engolir todo erro.

MCPs

- **code-review-graph**: avaliar rodar `generate_wiki_tool/run_postprocess_tool` para trazer `skills/specs` para dentro da camada de documentação do grafo, fechando a lacuna do achado 1.6.

3. Pontos de otimização de tokens

Onde a redundância é real

O conteúdo-fonte por projeto (`dossie_insumos.md` + `brief_criativo.json` + `config_projeto.json`, ~10KB no caso do Kit inLego) foi lido **por completo, do zero, 6 vezes** — uma por subagente-produtor-* — porque a REGRA 7 exige leitura integral desses artefatos, e cada subagente é uma unidade de fan-out isolada sem memória compartilhada. Essa redundância de I/O é real, mas individualmente barata (10KB não é caro). O custo mais alto observado nesta execução não foi reler texto, foi **retrabalho duplicado de diagnóstico**: o bug do `index.html` deletado por `compile-arte.py` (achado 1.4) foi descoberto e contornado de forma independente por 3 subagentes de revisão de marca diferentes, cada um gastando ferramentas de diagnóstico + reconstrução manual para confirmar um problema que os outros já tinham visto.

Avaliação honesta da proposta do usuário (grafo de insumos via `code-review-graph`)

Não funciona exatamente como descrito. `code-review-graph` é um grafo baseado em Tree-sitter sobre **código** — funções, chamadas, imports — não um índice semântico de conteúdo de marketing. Não existem “nós” de claims de produto ou de tom de voz para os redator-* consultarem; pedir isso ao grafo seria usar uma ferramenta fora do que ela foi construída para fazer. Mais importante: mesmo que esse índice existisse, a **REGRA 7 proíbe explicitamente resumir/truncar/grepar** `texto_base`, `brief_criativo.json` e o estado de pipeline — substituir a leitura integral por consulta a um grafo violaria essa regra diretamente, porque a fidelidade de marca (REGRA 6) depende de contexto completo, não de recuperação por similaridade.

Versão viável da ideia: usar o grafo para o que ele genuinamente faz bem — navegação **estrutural do pipeline** (`get_impact_radius_tool/query_graph_tool` sobre `scripts/compile-*.py`). Rodar essa consulta *antes* do fan-out de produção teria identificado que 3 compiladores diferentes compartilhavam o mesmo padrão de path hardcoded, evitando que o problema fosse descoberto 3 vezes durante a execução real. A economia de tokens, nesse caso, vem de detectar problemas de infraestrutura cedo — não de substituir a leitura de conteúdo de marca.

Otimizações concretas propostas

1. **Pre-flight de compatibilidade de slug (ganho alto, risco baixo).** Um script determinístico, rodado uma única vez antes do fan-out de produção, que verifica se `scripts/compile-*.py` contém qualquer string de slug hardcoded incompatível com o projeto atual (o padrão de grep usado nesta auditoria — `kit-start-flex/kit_start_flex` fora de `output/kit-start-flex/`) e falha cedo. Teria evitado 3 descobertas redundantes do mesmo bug nesta única execução.
2. **Consolidar revisão de marca em menos lotes quando o total de materiais é pequeno (ganho moderado, risco baixo).** Com 6 materiais, rodar `revisor-marca` em 1 lote de até 6 em vez de 2 lotes de 4+2 reduz overhead de reconstrução de contexto por subagente, sem abrir mão da REGRA 7 (cada subagente continua lendo tudo — só há menos subagentes no total).
3. **Digest estático do design system fixo (ganho baixo-moderado, risco baixo).** Ver recomendação de Skills na Seção 2 — não é insumo de projeto, então resumir/cachear esse conteúdo específico não conflita com a REGRA 7.

4. Verificação de consistência do pipeline como efeito colateral de economia de tokens.

O mesmo script de checklist proposto na Seção 2 (que pegaria o bug de `esbocar.md/textos`) também evita, em projetos futuros, o cenário de um operador selecionar um material via configuração manual que a interview não suporta, gerando uma rodada inteira de fan-out fadada a inconsistência.

Não recomendado: gerar um resumo/digest do dossiê de insumos para substituir a leitura integral pelos redator-*. Economizaria tokens, mas violaria a REGRA 6/7 diretamente — a fidelidade de claim-a-claim depende de cada redator ver o texto-base completo, não um resumo produzido por outro agente.

Resumo executivo

Prioridade	Item	Esforço estimado
Crítico	Adicionar “Textos de Apoio” à Rodada 4 de <code>esbocar.md</code>	Baixo
Alto	3 bugs de path hardcoded em <code>compile-html.py/compile-arte.py/compile-pdf.py</code>  Já corrigidos nesta sessão	
Alto	<code>compile-arte.py</code> deletando <code>index.html</code> exigido pelo validador  Já corrigido nesta sessão	
Médio	Cobertura do grafo de conhecimento vs. mandato da REGRA 9	Médio
Médio	Documentação de hooks desalinhada (<code>.gemini/</code> vs <code>.claude/settings.json</code>)	Baixo
Baixo	Permissões locais e scripts de debug escopados a <code>kit-start-flex</code>	Baixo
Otimização	Script de pre-flight de compatibilidade de slug antes do fan-out	Baixo
Otimização	Checklist de consistência pipeline ↔ entrevista	Baixo-Médio